



Prerequisites

CEN429 Secure Programming — 2026-2027 Fall

Asst. Prof. Dr. Uğur CORUH

What this course expects from you

- From the first week you **build and run C/C++ programs on your own computer**
- You run your project on **Git and GitHub**
- You prove your security controls with **unit tests**
- The formal prerequisite of the course is **CEN107 Algorithms and Programming I**

Plan of this page

1. What we bring from Algorithms and Programming I (required)
2. C programming basics (required)
3. Background briefly recalled in class
4. What must be installed on your computer
5. Check yourself (before the first class)

Rule: the development environment, Git, unit testing and template usage taught in the first weeks of CEN107 are **assumed** in this course.

Before the first class

Do the checks in Section 5 on your own computer.

If you get stuck on a step, go back to the corresponding Algorithms and Programming I page; if you still cannot solve it, ask in the first class.

1. What we bring from Algorithms and Programming I

- These four topics were taught in **CEN107 Algorithms and Programming I** (formerly CE103)
- They are **not retaught** here; they are used directly, assumed known
- Optional starting point: CEN107 (CE103) Week 1 — Introduction and developer roadmap

Development environment

Where is it taught? CEN107 (CE103) Week 2 — Development environments

- Compiler: GCC / Clang / MSVC
- IDE, WSL on Windows
- Building with CMake

Where is it used in this course? Every week's demos; lab setup in Week 1

Git and GitHub

Where is it taught? CEN107 (CE103) Week 3 — Version management with Git

- Creating a repository, `clone`, `commit`
- Branches, pull requests
- `.gitignore`

Where is it used in this course? Project guide (plan, setup, submission); change management in Week 1; version identity in Week 12

Unit testing and coverage tools

Where is it taught? CEN107 (CE103) Week 4 — Unit testing and libraries

- Writing tests
- Running them with `ctest`
- Coverage reports

Where is it used in this course? S16 test results in the project; sanitizers and fuzzing in Week 4; test plan in Week 12

Using project templates

Where is it taught? CEN107 (CE103) Weeks 2–4

- Forking a template
- Building it
- Producing its tests and documentation

Where is it used in this course? Project guide · 2. Project setup — template:

`cpp-cmake-ctest-template`

2. C programming basics (required)

Almost all demos are written in C, and security bugs mostly come from misusing these basic constructs. You should be able to read and write the following comfortably:

- Pointers, arrays and strings (`char[]` , the `\0` terminator)
- Dynamic memory (`malloc` , `calloc` , `free`) and structures (`struct`)
- Reading and writing files (`fopen` , `fread` , `fgets`)
- Compiling a program with several source files and using command-line arguments (`argc` , `argv`)

Used heavily in: **Week 1** (memory, overflows) and **Week 4** (C/C++ hardening)

3. Background briefly recalled in class

It is fine if you do not know these yet.

The "**Before we start**" section of the relevant week gives a short recap; looking at them in advance makes the class easier.

Terminal and number systems

- **Terminal:** `cd`, `ls`, file paths and environment variables in PowerShell and Linux/WSL
— needed in: **Week 1** and all demos
- **Binary and hexadecimal numbers, bytes, XOR**
— needed in: **Weeks 3, 9, 11**

Networking and other basics

- **Basic networking:** client–server, TCP/IP, HTTP
— needed in: **Weeks 3 and 10**
- **Simple SQL** (`SELECT ... WHERE`), basic **Java or Python** syntax
— needed in: **Week 5**

4. What must be installed on your computer

A laptop is required. Come to the first class with these tools installed:

- A C/C++ compiler: **Visual Studio 2022** (Desktop development with C++) on Windows, **GCC** or **Clang** on Linux/WSL
- **CMake**, **Git** and a **GitHub** account
- **WSL2 + Ubuntu** if you use Windows

What must be installed (continued)

- **OpenSSL 3, SQLite, Python 3, JDK 21**

GoogleTest, SoftHSM2 and Tigress are also used during the term; their installation is explained in the relevant week.

Setup guide for the demo code: [code/README.en.md](#)

5. Check yourself

Before the first class

Each of the commands below should print a version number (the numbers may differ).

Windows (PowerShell)

```
git --version      # git version 2.x
cmake --version    # cmake version 3.2x or later
wsl --status       # Default Distribution: Ubuntu
```

WSL / Linux

```
git --version          # git version 2.x
cmake --version        # cmake version 3.2x or later
gcc --version          # gcc (Ubuntu ...) 11 or later
openssl version        # OpenSSL 3.x
```

Try the template

Fork the project template to your own account, build it and run its tests:

```
git clone https://github.com/<your-username>/cpp-cmake-ctest-template.git
cd cpp-cmake-ctest-template
cmake -S . -B build
cmake --build build
ctest --test-dir build      # expected: 100% tests passed
```

Glossary

Term	Meaning
WSL	Windows Subsystem for Linux — running Linux inside Windows
CMake	Cross-platform build-system generator
CTest	The test runner that ships with CMake
Fork	Making your own copy of a repository
Coverage	A measure of how much of the code the tests exercise

Are you ready?

If all of these steps finish without errors, you are ready for the course.

Course site: ucoruh.github.io/cen429-secure-programming